information, which needs to be masked. Pandas offers ways to handle bad data by using methods like cleaning, dropping, replacing, masking, etc.

a. Empty rows can be removed with the *df.dropna(inplace=True)* operation.
b. Empty values can be replaced with *df.fillna(<value>, inplace=True)*. We can specify the column name to be placed in a particular column.
c. You can mask the values for sensitive and non-public data for all items NOT satisfying the condition *my_list.where(my_list < 5)*. Masking of values satisfying the condition can be done with *my_list.mask(my_list < 5)*.
d. Duplicates can be dropped from a dataframe using:

```
df.drop_
duplicates('<column>', keep = False)
df.drop_
duplicates('<column>', keep = 'first')
df.drop_
duplicates('<column>', keep = 'last')
```

Data analysis using Pandas

Table 2 lists the various functions in Pandas that perform data analysis as well as the syntax for usage. (*Note:* df stands for dataframe.)

Table 2

Function	Description	Syntax
Head	*Head()* function returns the first five rows	*df.head(x)*
tail	*tail()* function returns the last five rows by default	*df.tail(x)*
Loc	*Loc* function returns a particular row. Slicing of the data is also possible	*loc(x:y)*
Groupby	Groups data on a particular column	*groupby('<column>')*
Sum	Sum of values in a particular column	*df['column'].sum()*
Mean	Average of values in a particular column	*df['column']. mean()*
Min	Minimum value in a particular column	*df['column'].min()*
Max	Maximum value in a particular column	*df['column'].max()*
Sort	Sorts dataframe in a column	*df.sort_values(['column'])*
Size	Rows * columns	*df.size*
Describe	Describes details of the dataframe	*df.describe*
Crosstab	Creates a frequency tabulation of rows and columns	*pd.crosstab(df['column1'], df['column2'], margins = True)*
Duplicated	Returns *True* or *False* based on duplicate values in Column1 and Column2	*df.duplicated([column1, 'column2'])*

Advantages of Pandas

- It supports multi-index (hierarchical index) used for easy analysis of data having a large number of dimensions.
- It supports the creation of pivot tables, stack and unstack operations.
- Categorical data containing finite values can be processed with Pandas.
- It supports grouping and aggregations.
- Sorting can be explicitly disabled.
- It supports filtering at both row-level (gets the rows satisfying the filter condition) and column-level (selects only the required columns).
- Helps in reshaping of data sets. You can also transpose the values of the array and convert to a *List*.
- When you are processing data using Python, you can convert the Pandas dataframe to a multi-dimensional NumPy array; the values member variable is used for this.
- Supports label-oriented slicing of data.

The disadvantages

The code and syntax of Pandas is different from Python, which leads to a steep learning curve for some users. Also, a few concepts like three dimensional data are better handled in other libraries like NumPy.

Pandas really elevates the data analysis process in an efficient manner. Its compatibility with other libraries makes it very conducive for use in various scenarios. END

Reference

https://pandas.pydata.org

By: Phani Kiran

The author has over 18 years of experience in the IT industry and has worked on Big Data technologies.